

The Design and Security Analysis of the CSX AEAD Stream Cipher

John G. Underhill

Quantum Resistant Cryptographic Solutions Corporation
contact@qrcscorp.ca

September 30, 2025

Abstract

CSX is a keyed, permutation-based authenticated encryption construction that combines a 40 round ARX permutation over a 1024 bit state with a cSHAKE-derived subkey schedule and a KMAC-512 authentication layer. This paper specifies the CSX single-call AEAD construction, formalizes its security properties, and analyzes its resistance to established classes of cryptanalytic attacks. The specification is aligned with the reference implementation profile: the 1024 bit state is initialized from a cSHAKE-derived cipher subkey, fixed constants, and a 128 bit nonce that is advanced as the block counter; keystream generation uses modular-addition feed-forward from the permutation output; and authentication is provided by KMAC-512 over a canonical transcript containing associated data, nonce/counter material, ciphertext, and encoded lengths.

A sequence of game-based reductions bounds the IND-CCA advantage of any interval-respecting adversary by the PRP advantage against the ARX permutation, the strong unforgeability advantage against KMAC-512, the cSHAKE domain-separation advantage, and a permutation-switching term over the 1024 bit state. The analysis also states the operational assumptions required by the construction, including counter-interval disjointness under a fixed key and rejection before plaintext release in the single-call decryption interface. The cryptanalytic discussion treats differential, linear, algebraic, and structural considerations as design evidence rather than as exhaustive proof, and interprets post-quantum strength in the Q1 model through standard Grover-style key-search bounds. Under these stated assumptions, CSX provides the expected AEAD confidentiality and integrity guarantees for its specified single-call interface.

Contents

1	Introduction	4
2	Engineering Specification of CSX	5
2.1	High Level AEAD Interface	5
2.2	State and Parameter Layout	6
2.3	Key Expansion and MAC Key Derivation	6
2.4	ARX Permutation and keystream Generation	7
2.5	MAC Domain and Transcript Encoding	8
2.6	Canonical Pseudo-code for CSX	9
3	Formal Model	12
3.1	Notation and Conventions	12
3.2	CSX as an AEAD Scheme	13
3.3	Assumptions on Underlying Primitives	14
4	Security Definitions	15
4.1	IND CPA Security	15
4.2	IND CCA Security	16
4.3	Ciphertext Integrity (INT CTXT)	16
4.4	Key Recovery and Related Key Security	17
4.5	Post-Quantum Adversarial Model	17
5	Security Theorems and Reductions	18
5.1	AEAD Security Theorem	18
5.2	Reduction for Confidentiality	19
5.3	Reduction for Integrity	21
5.4	Post-Quantum Security Bounds	21
6	Cryptanalytic Evaluation	22
6.1	Differential and Linear Analysis	22
6.2	Algebraic and Integral Attacks	24
6.3	Reduced Round Analysis and Security Margin	24
6.4	Attacks on the cSHAKE and KMAC Components	25
7	Validation Artifacts and Sanity Checks	26
7.1	Keystream Statistical Tests	26
7.2	Diffusion and Avalanche Measurements	26
8	Discussion and Limitations	27

8.1	Modeling Assumptions and Boundaries	27
8.2	Limitations and Open Questions	28
9	Conclusion	29

1 Introduction

Authenticated encryption with associated data (AEAD) provides a unified mechanism for ensuring both confidentiality and integrity of transmitted or stored information. Modern AEAD designs typically integrate a stream or block cipher with an authentication component, often relying on well-studied families such as AES-GCM, ChaCha20-Poly1305, or Ascon, the NIST Lightweight Cryptography winner. In settings where long-term security, large state widths, or post-quantum design considerations are desired, alternative constructions remain an active area of research.

CSX is a wide state AEAD construction that combines a 40 round ARX permutation over a 1024 bit internal state with subkeys derived through cSHAKE-512 and a KMAC-512 authentication layer. The cipher follows an Encrypt then MAC structure and operates in a deterministic counter based mode in which keystream blocks are generated by applying the permutation to a state containing a 128 bit counter, fixed domain separation constants, and the cSHAKE derived subkey. The authentication key is produced by an additional cSHAKE squeeze step, ensuring computational independence between the encryption and authentication components. CSX is intended to provide strong diffusion, a conservative round budget, and post-quantum resilience through a 512 bit master key. This paper presents a formal design and security analysis of CSX. First, we specify the implementation profile analyzed in this paper, expressed in implementation-agnostic mathematical terms. This specification defines the state layout, key expansion procedure, ARX permutation structure, counter behavior, and authenticated transcript format. The algorithms presented here define the CSX single-call AEAD construction and form the basis for all subsequent formal analysis.

Next, we model CSX within a standard AEAD security framework and formalize the IND-CPA, IND-CCA, and INT-CTXT security notions relevant to the construction. Using game-based reductions, we show that the IND-CCA advantage of any adversary is bounded by the pseudo-random permutation advantage against the 40 round ARX permutation and the strong unforgeability advantage against KMAC-512, together with a permutation switching term and negligible interaction terms introduced by cSHAKE domain separation. These reductions justify the confidentiality and integrity guarantees of CSX under widely used assumptions on its underlying primitives. We then examine the cryptanalytic properties of the 1024 bit ARX permutation, including its differential, linear, algebraic, and structural behavior. Reduced round analysis is used to assess the security margin, and findings are compared to published bounds for ARX based designs of similar structure. Consideration is also given to quantum adversaries, for whom Grover-style search reduces effective key strength by a square root factor. The choice of a 512 bit master key in CSX is shown to align with a 256 bit post-quantum target.

Finally, we provide empirical sanity checks concerning implementation behavior and diffusion characteristics. These results complement the formal analysis but are not relied upon as primary evidence of security; detailed throughput comparisons against established AEAD constructions are left to a separate performance study. This work is a design and security analysis of CSX. All security claims are explicitly tied to the assumptions stated in the model and supported by the engineering specification and implementation behavior. The remainder of the paper is structured as follows. Section 2 presents the engineering specification of CSX.

Section 3 formalizes the model and assumptions. Section 4 defines the security notions used throughout the paper. Section 5 provides the main theorems and reductions. Section 6 examines cryptanalytic properties of the permutation and MAC components. Section 7 gives empirical sanity checks. Section 8 discusses limitations and Section 9 concludes.

2 Engineering Specification of CSX

This section specifies the CSX single-call AEAD construction analyzed in this paper. The description is aligned with the reference implementation profile (`csx.c`, `csx.h`) and defines the algorithmic behavior required for the formal model. All subsequent security definitions and reductions refer to the single-call interface defined here; the internal helper `qsc_csx_extended_transform` is not part of the CSX specification and is excluded from the AEAD model.

2.1 High Level AEAD Interface

CSX is an authenticated encryption scheme with associated data (AEAD) defined by a pair of deterministic algorithms

$$\text{Enc} : \mathcal{K} \times \mathcal{N} \times \mathcal{A} \times \mathcal{P} \rightarrow \mathcal{C} \times \mathcal{T}, \quad \text{Dec} : \mathcal{K} \times \mathcal{N} \times \mathcal{A} \times \mathcal{C} \times \mathcal{T} \rightarrow \mathcal{P} \cup \{\perp\},$$

where:

- $\mathcal{K} = \{0, 1\}^{512}$ is the 512 bit master key space,
- $\mathcal{N} = \{0, 1\}^{128}$ is the 128 bit nonce space,
- $\mathcal{A}, \mathcal{P} \subseteq \{0, 1\}^*$ are associated data and plaintexts,
- $\mathcal{C} = \{0, 1\}^{|P|}$ is the ciphertext space,
- $\mathcal{T} = \{0, 1\}^{512}$ is the 512 bit authentication tag space.

Encryption takes (K, N, A, P) and outputs (C, T) , where C is the XOR of P with a keystream derived from a 1024 bit ARX permutation, and T is a KMAC-512

tag over a canonical transcript. Decryption recomputes the tag from (K, N, A, C) and returns P only if the tag matches; otherwise it returns $\perp$.

CSX is deterministic: for fixed (K, N, A, P) the pair (C, T) is uniquely determined. Counter-interval disjointness under each key is required for confidentiality.

2.2 State and Parameter Layout

CSX maintains a 1024 bit internal state represented as sixteen 64 bit words:

$$S = (S[0], S[1], \dots, S[15]), \quad S[i] \in \{0, 1\}^{64}.$$

The words are interpreted in little-endian order, matching the reference implementation. The state is arranged as a 4×4 matrix of 64 bit lanes, in the manner of ChaCha. The state layout is defined as follows.

- **Words 0-7:** the 512 bit encryption subkey K_c derived from cSHAKE-512 (matrix rows 0 and 1).
- **Words 8-11:** the first four 64 bit words $(C_0, \dots, C_3)$ of the fixed 48 byte domain constant block (matrix row 2).
- **Words 12-13:** the 128 bit nonce loaded as two 64 bit little-endian words $S[12] = N_0$ and $S[13] = N_1$. These two words are the active block-counter region. After each keystream block they are incremented as a 128 bit little-endian integer,

$$(S[12], S[13]) \leftarrow (S[12], S[13]) + 1 \bmod 2^{128}.$$

- **Words 14-15:** the remaining two 64 bit words (C_4, C_5) of the fixed constant block. These words remain fixed during ordinary keystream generation and provide additional domain separation inside the permuted state.

Thus the external nonce is the initial value of the 128 bit block counter. A message of σ 1024 bit blocks initialized at nonce N consumes the interval $[N, N + \sigma)$ in $\mathbb{Z}/2^{128}\mathbb{Z}$. Security therefore requires pairwise disjoint counter intervals under a fixed key, not merely syntactic nonce non-repetition. Uniform random 128 bit nonces make accidental interval overlap negligible for realistic workloads; deliberate sequential allocation must advance by at least the maximum number of blocks that any message can consume.

2.3 Key Expansion and MAC Key Derivation

CSX uses a single cSHAKE-512 instance for all subkey generation.

Let $\text{cSHAKE512}(X, \text{name}, \text{custom})$ denote cSHAKE-512 absorbed with input X

and domain separated by the function name string *name* and customization string *custom*.

Given a master key K and optional customization string *info*, key expansion proceeds as follows:

1. Initialize a cSHAKE-512 sponge with input K and customization *info*.
2. Squeeze 512 bits to obtain the encryption subkey K_c .
3. Force an additional permutation by requesting further output.
4. Squeeze 512 bits from the next block to obtain the MAC key K_{mac} .

Because K_c and K_{mac} are read from disjoint 512 bit segments of the cSHAKE-512 output stream, they are independent and uniformly distributed under the random oracle / indistinguishability model for the Keccak sponge. The intervening squeeze in step 3 is the ordinary sponge output step that separates the two output blocks; it is the disjointness of the output segments, rather than the squeeze itself, that yields independence.

2.4 ARX Permutation and keystream Generation

Let $\text{Perm}(S)$ denote the 40 round ARX permutation applied to a 1024 bit state. Structurally, Perm is a 64 bit word variant of the ChaCha permutation: the sixteen 64 bit words are arranged as a 4×4 matrix, and each pair of rounds applies four column mixes ($G_0, \dots, G_3$) followed by four diagonal mixes ($G_4, \dots, G_7$), exactly as detailed in Table 1. Each round consists of a fixed sequence of 64 bit operations:

$$\begin{aligned} &\text{for } (i, j, k, r) \text{ in a fixed schedule:} \\ &\quad S[i] \leftarrow (S[i] + S[j]) \bmod 2^{64}; \\ &\quad S[k] \leftarrow S[k] \oplus S[i]; \\ &\quad S[k] \leftarrow \text{ROTL}_r(S[k]), \end{aligned}$$

where ROTL_r denotes a left rotation by r bits. The schedule and rotation constants correspond exactly to those encoded in the reference implementation.

For a given state S , the permutation output is combined with the original state by component-wise modular-addition feed-forward:

$$K_{\text{blk}}[i] = \text{Perm}(S)[i] + S[i] \bmod 2^{64}, \quad 0 \leq i < 16,$$

which we write compactly as $K_{\text{blk}} = \text{Perm}(S) \boxplus S$. This produces a 1024 bit keystream block.

The counter words ($S[12], S[13]$) are incremented as a 128 bit little-endian integer modulo 2^{128} after each block:

$$ctr \leftarrow (ctr + 1) \bmod 2^{128}.$$

To encrypt plaintext P , the 1024 bit keystream block is truncated to $|P_i|$ for each segment P_i , and ciphertext blocks are computed as

$$C_i = P_i \oplus K_{\text{blk},i}.$$

ARX Mixing Schedule

The 1024 bit CSX permutation operates on state words $X_0, \dots, X_{15}$ and applies, in each pair of rounds, eight ARX mixing functions on word quadruples. Each mixing function has the generic structure

$$(a, b, c, d) \mapsto (a', b', c', d')$$

implemented as a sequence of additions, XORs, and rotations with four rotation constants (r_0, r_1, r_2, r_3) , as in the reference implementation `csx_permute_p1024c`. The word tuples and rotation constants for the two round pattern are summarised in Table 1.

Mix	Word tuple (a, b, c, d)	Rotations (r_0, r_1, r_2, r_3)
<i>Round n (column mixes)</i>		
G_0	(X_0, X_4, X_8, X_{12})	$(38, 19, 10, 55)$
G_1	(X_1, X_5, X_9, X_{13})	$(33, 4, 51, 13)$
G_2	$(X_2, X_6, X_{10}, X_{14})$	$(16, 34, 56, 51)$
G_3	$(X_3, X_7, X_{11}, X_{15})$	$(4, 53, 42, 41)$
<i>Round $n+1$ (diagonal mixes)</i>		
G_4	$(X_0, X_5, X_{10}, X_{15})$	$(34, 41, 59, 17)$
G_5	$(X_1, X_6, X_{11}, X_{12})$	$(23, 31, 37, 20)$
G_6	(X_2, X_7, X_8, X_{13})	$(31, 44, 47, 46)$
G_7	(X_3, X_4, X_9, X_{14})	$(12, 47, 44, 30)$

Table 1: ARX mixing schedule for the CSX 1024 bit permutation as implemented in `csx_permute_p1024c`. Each loop iteration applies G_0 to G_7 once, implementing two rounds and subtracting 2 from `ctr`.

2.5 MAC Domain and Transcript Encoding

CSX authenticates a canonical transcript using KMAC-512 keyed with K_{mac} . Let $\text{KMAC}_{512}(K_{\text{mac}}, M)$ denote the 512 bit KMAC output on message M .

For the single-call interface, the transcript is the exact byte string absorbed by the reference implementation. Associated data is supplied before the transform

call through `qsc_csx_set_associated`; that function absorbs the associated data and then a 32 bit little-endian encoding of its length only when the associated-data length is nonzero. The transform then absorbs the 16 byte nonce value stored at call entry, the ciphertext, and a 64 bit little-endian encoding of the processed ciphertext length. Consequently,

$$\tau(A, N, C) = \begin{cases} A \parallel \text{le32}(|A|) \parallel N \parallel C \parallel \text{le64}(|C|), & |A| > 0, \\ N \parallel C \parallel \text{le64}(|C|), & |A| = 0. \end{cases}$$

The lengths are byte lengths and N is the 16 byte value of $(S[12], S[13])$ before the first block of the single-call transform is processed. The authentication tag is

$$T = \text{KMAC}_{512}(K_{\text{mac}}, \tau(A, N, C)).$$

Encryption follows Encrypt-then-MAC ordering: ciphertext is generated first, and the tag is computed over the ciphertext transcript. Single-call decryption recomputes the tag and compares it before invoking the counter-mode transform, so plaintext is not produced on authentication failure. The internal helper `qsc_csx_extended_transform` is excluded from this specification and from the security theorems.

2.6 Canonical Pseudo-code for CSX

The following Pseudo-code defines CSX in a language neutral manner. The pseudo-code matches the behavior of the reference implementation and serves as the authoritative algorithmic description for the security analysis.

Algorithm 1 CSX_Initialize

```
1: input master key  $K \in \{0, 1\}^{512}$ , nonce  $N \in \{0, 1\}^{128}$ , optional info string  $info$ 
2: output initialized context  $ctx$ 
3: // Derive the cSHAKE name string
4: if  $info$  is empty then
5:    $nme \leftarrow csx\_name$  // fixed domain string of length CSX_NAME_SIZE
6: else
7:    $nme \leftarrow$  first CSX_NAME_SIZE bytes of  $info$  (zero padded if shorter)
8: end if
9: // Initialize cSHAKE-512 with the master key and name
10:  $X \leftarrow cSHAKE512.Init(K, nme, \varepsilon)$ 
11: // First squeeze: cipher subkey
12:  $buf \leftarrow cSHAKE512.Squeeze(X, 512)$ 
13:  $K_c \leftarrow$  first 64 bytes of  $buf$ 
14: // Load the CSX permutation state from key, nonce/counter, and constants
15: Let  $csx\_info \in \{0, 1\}^{384}$  be the fixed 48 byte domain constant block  $(C_0, \dots, C_5)$ 
16:  $ctx.state[0..7] \leftarrow K_c$ 
17:  $ctx.state[8..11] \leftarrow csx\_info[0..31]$  // constants  $C_0..C_3$ 
18:  $ctx.state[12..13] \leftarrow N$  // initial 128 bit counter value
19:  $ctx.state[14..15] \leftarrow csx\_info[32..47]$  // constants  $C_4, C_5$ 
20: // Second squeeze: MAC key
21:  $buf' \leftarrow cSHAKE512.Squeeze(X, 512)$ 
22:  $K_{mac} \leftarrow$  first 64 bytes of  $buf'$ 
23: // Initialize the KMAC-512 state with  $K_{mac}$ 
24:  $ctx.kstate \leftarrow KMAC512.Init(K_{mac}, kmac\_domain)$ 
25:  $ctx.counter \leftarrow 0$  // processed-bytes counter; distinct from  $S[12], S[13]$ 
26: return  $ctx$ 
```

Algorithm 2 CSX_Permute

```
1: input state  $S[0..15]$ 
2: for  $r = 0$  to 39 do
3:   for each tuple  $(i, j, k, \rho)$  in the round schedule do
4:      $S[i] \leftarrow (S[i] + S[j]) \bmod 2^{64}$ 
5:      $S[k] \leftarrow S[k] \oplus S[i]$ 
6:      $S[k] \leftarrow \text{ROTL}_{\rho}(S[k])$ 
7:   end for
8: end for
9: return  $S$ 
```

Algorithm 3 CSX_Encrypt

```
1: input  $K, N, A, P$ 
2: output  $C, T$ 
3: Initialize ctx by calling CSX_Initialize( $K, N, info$ )
   // derives  $K_c, K_{\text{mac}}$ ; loads state; inits KMAC
4: Let  $S \leftarrow \text{ctx.state}[0..15]$  and let  $K_{\text{mac}}$  be the MAC key established by
   CSX_Initialize
5:  $C \leftarrow \varepsilon$ 
6: for each message segment  $P$  processed under a fixed context ctx do
   // keystream generation and counter management occur inside
   CSX_Transform.
7:    $(C, \text{ctx}) \leftarrow \text{CSX\_Transform}(\text{ctx}, P)$ 
8: end for
9:  $M \leftarrow A \parallel \text{le32}(|A|) \parallel N \parallel C \parallel \text{le64}(|C|)$ 
10:  $T \leftarrow \text{KMAC}_{512}(K_{\text{mac}}, M)$ 
11: return  $(C, T)$ 
```

Algorithm 4 CSX_Decrypt

```
1: input  $K, N, A, C, T$ 
2: output  $P$  or  $\perp$ 
3: Initialize  $\text{ctx}$  by calling  $\text{CSX\_Initialize}(K, N, \text{info})$ 
   // derives  $K_c, K_{\text{mac}}$ ; loads state
4: Let  $S \leftarrow \text{ctx.state}[0..15]$  and let  $K_{\text{mac}}$  be the MAC key established by
   CSX_Initialize
5:  $M \leftarrow A \parallel \text{le32}(|A|) \parallel N \parallel C \parallel \text{le64}(|C|)$ 
6:  $T' \leftarrow \text{KMAC}_{512}(K_{\text{mac}}, M)$ 
7: if  $T' \neq T$  then
8:   return  $\perp$  // reject before any plaintext is produced
9: end if
10:  $P \leftarrow \varepsilon$ 
11: for each ciphertext block  $C_i$  do
12:    $K_{\text{blk}} \leftarrow \text{CSX\_Permute}(S) \boxplus S$ 
13:    $P_i \leftarrow C_i \oplus K_{\text{blk},i}$ 
14:    $P \leftarrow P \parallel P_i$ 
15:   increment the block counter words  $S[12], S[13]$  by 1 mod  $2^{128}$ 
16: end for
17: return  $P$ 
```

3 Formal Model

This section formalizes the CSX construction introduced in Section 2. We specify the notation, the algorithmic structure of CSX as an AEAD scheme, and the assumptions under which the subsequent security analysis is carried out. All definitions are stated in a manner compatible with standard cryptographic treatment of authenticated encryption primitives.

3.1 Notation and Conventions

Let $\{0, 1\}^n$ denote the set of bit-strings of length n and let $\{0, 1\}^*$ denote the set of all finite bit-strings. For bit-strings X and Y , we write $X \parallel Y$ for concatenation and $X \oplus Y$ for bitwise XOR over equal length strings. For an integer m , we write $\text{le32}(m)$ and $\text{le64}(m)$ for the 32 bit and 64 bit little endian encodings of m , respectively.

The spaces used throughout this work are:

$$\mathcal{K} = \{0, 1\}^{512} \quad \mathcal{N} = \{0, 1\}^{128} \quad \mathcal{A}, \mathcal{P} \subseteq \{0, 1\}^* \quad \mathcal{T} = \{0, 1\}^{512}.$$

The CSX internal state is a 1024 bit quantity represented as

$$S = (S[0], \dots, S[15]), \quad S[i] \in \{0, 1\}^{64},$$

interpreted in little endian form as specified in the engineering description.

Underlying Primitives. We model the following components abstractly:

- **ARX Permutation.** Let $\text{Perm} : \{0, 1\}^{1024} \rightarrow \{0, 1\}^{1024}$ denote the 40 round ARX permutation described in Section 2.4. The corresponding keystream block function is defined as

$$\text{KS}(S) = \text{Perm}(S) \boxplus S.$$

Here $\boxplus$ denotes component-wise addition modulo 2^{64} .

- **cSHAKE-512.** The key expansion procedure uses an instance of cSHAKE-512 to derive two computationally independent 512 bit keys (K_c, K_{mac}) from a 512 bit master key and optional customization data. The output is modeled as arising from a random oracle with domain separation as specified in SP 800-185.
- **KMAC-512.** The authenticated transcript is processed using KMAC-512. We write $\text{KMAC}_{512}(K_{\text{mac}}, M)$ for the 512 bit MAC tag on message M under key K_{mac} .

All algorithms and adversaries are probabilistic unless stated otherwise.

3.2 CSX as an AEAD Scheme

CSX is defined as a pair of deterministic algorithms (Enc, Dec) operating over the spaces defined above. For $(K, N, A, P) \in \mathcal{K} \times \mathcal{N} \times \mathcal{A} \times \mathcal{P}$, encryption proceeds as follows.

1. Compute (K_c, K_{mac}) using the key expansion algorithm of Section 2.3.
2. Initialize the 1024 bit state with K_c , the fixed constant block, the nonce N in words 12-13 as the initial block-counter value, and the constant words (C_4, C_5) in words 14-15.
3. For each 1024 bit segment of P , compute a keystream block

$$K_{\text{blk}} \leftarrow \text{KS}(S),$$

XOR it with the plaintext segment, append the resulting ciphertext block to C , and increment the 128 bit block counter in words $S[12], S[13]$.

4. Construct the authenticated transcript

$$M = A \parallel \text{le32}(|A|) \parallel N \parallel C \parallel \text{le64}(|C|).$$

5. Compute the authentication tag as

$$T = \text{KMAC}_{512}(K_{\text{mac}}, M).$$

6. Output (C, T) .

Decryption $\text{Dec}(K, N, A, C, T)$ recomputes (K_c, K_{mac}) , recomputes

$$T' = \text{KMAC}_{512}(K_{\text{mac}}, M),$$

and returns $\perp$ if $T' \neq T$. If the tags match, it regenerates the same keystream sequence and outputs the XOR of the keystream with C .

The scheme is *deterministic*: for fixed inputs (K, N, A, P) , the output (C, T) is unique. Confidentiality requires that the 128 bit counter intervals induced by N and the message length be pairwise disjoint under the same key K .

3.3 Assumptions on Underlying Primitives

The security guarantees established later in this paper rest on the following modeling assumptions.

ARX Permutation. We assume that the 40 round ARX permutation underlying CSX behaves as a pseudo-random permutation (PRP) on $\{0, 1\}^{1024}$ against any classical probabilistic polynomial time adversary. Reduced round analysis and cryptanalytic evidence supporting this assumption are provided in Section 6.

cSHAKE-512. We model the cSHAKE-512 instance used in key expansion as a domain separated random oracle, as per the indistinguishability guarantees of Keccak based sponge constructions. The two subkeys produced by distinct squeeze operations are therefore treated as independent uniformly random 512 bit strings.

KMAC-512. We assume KMAC-512 is a *strongly* unforgeable MAC (SUF-CMA). Since KMAC-512 is a fixed output length keyed sponge that is a PRF under the random permutation model for Keccak- $f[1600]$, strong unforgeability follows from its PRF security: no efficient adversary can produce a fresh (message, tag) pair, including a new tag on a previously authenticated message. Concretely,

for an adversary issuing q_t tagging queries and q_v verification queries, with a 512 bit tag and sponge capacity $c = 1024$,

$$\text{Adv}_{\text{KMAC}}^{\text{suf-cma}}(\mathcal{A}) \lesssim \frac{q_v}{2^{512}} + \frac{(q_t + q_v)^2}{2^{1024}},$$

where the first term is the probability of guessing the 512 bit tag of a fresh message across q_v verification attempts, and the second is the internal state-collision (capacity) term inherited from the sponge. For all practical query budgets the tag-guessing term $q_v/2^{512}$ dominates.

Quantum Adversaries. For quantum adversaries, this paper uses the Q1 convention: adversaries may perform quantum computation but interact with encryption and verification oracles through classical inputs and outputs. In this model, exhaustive search over a 512 bit symmetric key admits the usual Grover-style square-root speedup and therefore gives approximately 2^{256} brute-force resistance. A full Q2/superposition-query proof for the composed AEAD construction is outside the scope of this paper.

These assumptions are consistent with the current state of analysis for ARX based permutations and Keccak derived primitives. All formal security bounds established in Section 5 are expressed explicitly in terms of these assumptions.

4 Security Definitions

This section formalizes the confidentiality and integrity notions relevant to CSX as an authenticated encryption scheme. All definitions follow standard game-based frameworks for symmetric key cryptography. Unless otherwise stated, all adversaries are probabilistic polynomial time (PPT) algorithms.

4.1 IND CPA Security

Indistinguishability under chosen plaintext attack (IND-CPA) captures confidentiality of an encryption algorithm in the absence of decryption queries. Let $\mathcal{A}$ be an adversary making encryption queries to an oracle $\mathcal{O}_{\text{Enc}}$.

Experiment $\text{Exp}_{\text{CSX}}^{\text{ind-cpa}}(b)$ The bit $b \in \{0, 1\}$ determines which challenge message is encrypted.

1. A random key $K \leftarrow \mathcal{K}$ is sampled.

2. $\mathcal{A}$ is given oracle access to $\mathcal{O}_{\text{Enc}}$ defined as follows: for each query (N, A, P_0, P_1) with $|P_0| = |P_1|$ and $N \in \mathcal{N}$ not used in any prior query, the oracle returns

$$(C, T) \leftarrow \text{Enc}(K, N, A, P_b).$$

3. At the end of the experiment, $\mathcal{A}$ outputs a bit b' .

The IND-CPA advantage of $\mathcal{A}$ is

$$\text{Adv}_{\text{CSX}}^{\text{ind-cpa}}(\mathcal{A}) = |\Pr[b' = 1 \mid b = 1] - \Pr[b' = 1 \mid b = 0]|.$$

The oracle is interval-respecting: for every query it rejects any nonce whose induced counter interval $[N, N + \sigma)$ overlaps the interval of a previous encryption query under the same key, where σ is the number of 1024 bit keystream blocks consumed.

4.2 IND CCA Security

Indistinguishability under chosen ciphertext attack (IND-CCA) strengthens IND-CPA by allowing the adversary to query a decryption oracle, except on the challenge ciphertext.

Experiment $\text{Exp}_{\text{CSX}}^{\text{ind-cca}}(b)$

1. Sample $K \leftarrow \mathcal{K}$.
2. $\mathcal{A}$ receives oracle access to:
 - $\mathcal{O}_{\text{Enc}}$ as in the IND-CPA experiment;
 - $\mathcal{O}_{\text{Dec}}$ defined as $\mathcal{O}_{\text{Dec}}(N, A, C, T) = \text{Dec}(K, N, A, C, T)$, except that any query equal to the challenge tuple (N^*, A^*, C^*, T^*) is forbidden.
3. $\mathcal{A}$ outputs a bit b' .

The IND-CCA advantage is

$$\text{Adv}_{\text{CSX}}^{\text{ind-cca}}(\mathcal{A}) = |\Pr[b' = 1 \mid b = 1] - \Pr[b' = 1 \mid b = 0]|.$$

Because CSX implements explicit tag verification before releasing plaintext, the definition above matches the classical Encrypt then MAC security framework.

4.3 Ciphertext Integrity (INT CTXT)

Ciphertext integrity ensures that an adversary cannot produce a valid new ciphertext tag pair that decrypts to anything other than $\perp$, even after observing valid ciphertexts and receiving decryption responses.

Experiment $\text{Exp}_{\text{CSX}}^{\text{int-ctxt}}$

1. Sample $K \leftarrow \mathcal{K}$.
2. $\mathcal{A}$ is given oracle access to:
 - $\mathcal{O}_{\text{Enc}}$ returning $(C, T) = \text{Enc}(K, N, A, P)$,
 - $\mathcal{O}_{\text{Dec}}$ returning $\text{Dec}(K, N, A, C, T)$.
3. At the end, $\mathcal{A}$ outputs (N^*, A^*, C^*, T^*) .

A forgery succeeds if:

$$\text{Dec}(K, N^*, A^*, C^*, T^*) \neq \perp,$$

and (N^*, A^*, C^*, T^*) was never returned by $\mathcal{O}_{\text{Enc}}$.

The INT-CTXT advantage is

$$\text{Adv}_{\text{CSX}}^{\text{int-ctxt}}(\mathcal{A}) = \Pr[\text{forgery succeeds}].$$

Because CSX authenticates a transcript containing both ciphertext and lengths, replay, extension, and truncation forgeries are all covered by this definition.

4.4 Key Recovery and Related Key Security

Key recovery considers an adversary attempting to determine the master key K through chosen query access.

Key Recovery. Let $\mathcal{A}$ be an adversary with access to $\mathcal{O}_{\text{Enc}}$ and $\mathcal{O}_{\text{Dec}}$. The key recovery advantage is

$$\text{Adv}_{\text{CSX}}^{\text{krec}}(\mathcal{A}) = \Pr[\mathcal{A}^{\mathcal{O}_{\text{Enc}}, \mathcal{O}_{\text{Dec}}} \text{ outputs } K].$$

Related Key Model. CSX does not aim to provide related key security in models allowing adversaries to derive encryptions under systematically modified keys. We consider only the standard single key model, consistent with common AEAD usage. Related key attacks on the ARX permutation or on cSHAKE-512 are outside the intended threat model.

4.5 Post-Quantum Adversarial Model

In the quantum setting considered here, adversaries may run quantum algorithms but issue only classical oracle queries to Enc and Dec. This is the Q1 model. It is the most direct model for deployed AEAD APIs, whose inputs and outputs are classical byte strings.

Grover-Based Search. We assume the standard quadratic speedup for exhaustive key search: brute-force search over a 512 bit master key requires approximately 2^{256} quantum work.

Keccak-Derived Components. The cSHAKE-512 and KMAC-512 components are interpreted through the conventional sponge and keyed-sponge assumptions with the usual caution that a complete Q2 analysis would require superposition-query access to the underlying interfaces. This paper does not claim such a Q2 proof.

Post-Quantum Security Level. Under these Q1 assumptions, CSX targets approximately 256 bit post-quantum brute-force resistance. The formal bounds below should therefore be read as classical-oracle AEAD bounds whose key-search interpretation is adjusted by the standard Grover-style square-root rule.

These definitions provide the framework for the reductions and security theorems developed in Section 5.

5 Security Theorems and Reductions

This section establishes provable bounds for the confidentiality and integrity of CSX under the assumptions stated in Section 3. Three main results are presented: an IND-CCA bound for the AEAD construction, a confidentiality reduction to the ARX permutation and cSHAKE derived keys, and an integrity reduction to KMAC-512. Finally, we quantify the post-quantum security level inherited from the 512 bit master key.

5.1 AEAD Security Theorem

We begin by stating the combined AEAD security bound for CSX. Let an adversary $\mathcal{A}$ make at most q_e encryption queries and q_d decryption (verification) queries, and let σ denote the total number of 1024 bit keystream blocks produced across all encryption queries (the data complexity in blocks).

Theorem 1 (IND-CCA Security of CSX). *For any interval-respecting IND-CCA adversary $\mathcal{A}$ against CSX, there exist adversaries B_1 , B_2 , and B_3 against the underlying primitives, each running in time comparable to $\mathcal{A}$, such that*

$$\text{Adv}_{\text{CSX}}^{\text{ind-cca}}(\mathcal{A}) \leq \text{Adv}_{\text{Perm}}^{\text{prp}}(B_1) + \text{Adv}_{\text{KMAC}}^{\text{suf-cma}}(B_2) + \text{Adv}_{\text{cSHAKE}}^{\text{ro}}(B_3) + \frac{\sigma^2}{2^{1024}}.$$

The term $\sigma^2/2^{1024}$ is the permutation-to-random-function switching loss on the 1024 bit state and is negligible for any feasible σ . Constant factors arising from invoking the PRP and cSHAKE assumptions in both the confidentiality and integrity reductions are absorbed into B_1 and B_3 .

Remark 1 (Nonce generation). *The bound above assumes an interval-respecting adversary: no two encryption queries under the same key consume overlapping 128 bit counter intervals. If a deployment draws nonces uniformly at random instead of enforcing interval separation, an additional interval-overlap term must be included. For q_e queries consuming at most σ total 1024 bit blocks, a simple union bound gives an overlap term at most $q_e\sigma/2^{128}$. CSX is a counter-based scheme and provides no confidentiality guarantee under counter-interval overlap.*

The proof combines a confidentiality bound (Section 5.2) with an integrity bound (Section 5.3) through the Encrypt-then-MAC composition theorem of Bellare and Namprempre, which establishes that an encryption scheme that is IND-CPA secure, composed with a strongly unforgeable MAC in Encrypt-then-MAC order, is IND-CCA and INT-CTXT secure. Specifically,

$$\text{Adv}_{\text{CSX}}^{\text{ind-cca}}(\mathcal{A}) \leq \text{Adv}_{\text{CSX}}^{\text{ind-cpa}}(\mathcal{A}') + \text{Adv}_{\text{CSX}}^{\text{int-ctxt}}(\mathcal{A}''),$$

and the two terms are bounded by the hybrid arguments detailed below.

5.2 Reduction for Confidentiality

Let $\mathcal{A}$ be an IND-CPA adversary against CSX. We bound its advantage through a sequence of games

$$G_0 \rightarrow G_1 \rightarrow G_2 \rightarrow G_3$$

and bound the difference in adversarial advantage between each pair. The extension to IND-CCA is handled separately at the end of this subsection via Encrypt-then-MAC composition.

Game G_0 . This is the real IND-CPA experiment for CSX.

Game G_1 : Replace ARX permutation with a random permutation. In G_1 we replace Perm with a uniformly random permutation $\pi : \{0, 1\}^{1024} \rightarrow \{0, 1\}^{1024}$. The adversary's distinguishing advantage changes by at most

$$|\Pr[G_0 \Rightarrow 1] - \Pr[G_1 \Rightarrow 1]| \leq \text{Adv}_{\text{Perm}}^{\text{prp}}(B_1),$$

where B_1 is a PRP distinguisher constructed from $\mathcal{A}$.

Game G_2 : Replace K_c and K_{mac} with independent random keys. In G_2 we replace the outputs of cSHAKE-512 with independent uniform strings $(K_c, K_{\text{mac}}) \leftarrow \{0, 1\}^{512} \times \{0, 1\}^{512}$. By the domain separation of cSHAKE and its indistinguishability properties, the change in advantage is bounded by the random oracle advantage of cSHAKE:

$$|\Pr[G_1 \Rightarrow 1] - \Pr[G_2 \Rightarrow 1]| \leq \text{Adv}_{\text{cSHAKE}}^{\text{ro}}(B_3),$$

which is treated as negligible.

Game G_3 : Replace keystream with uniform random bits. In G_3 we replace each keystream block $K_{\text{blk}} = \pi(S) \boxplus S$ with a uniform 1024 bit string. Under interval-respecting queries, the initial nonce and the block counter in words $S[12], S[13]$ ensure that all σ permutation inputs produced across the experiment are pairwise distinct. By the permutation-to-random-function switching lemma on the 1024 bit domain, a uniformly random permutation π evaluated on σ distinct inputs is, together with the public modular-addition feed-forward $\pi(S) \boxplus S$, within statistical distance $\sigma^2/2^{1024}$ of σ independent uniform 1024 bit strings. Hence

$$|\Pr[G_2 \Rightarrow 1] - \Pr[G_3 \Rightarrow 1]| \leq \frac{\sigma^2}{2^{1024}}.$$

Conclusion. In G_3 the ciphertext is an XOR of the message with a uniform string and therefore carries no information about the challenge bit. Hence

$$\Pr[G_3 \Rightarrow 1] = \frac{1}{2}.$$

Summing the differences between hybrids gives the confidentiality bound

$$\text{Adv}_{\text{CSX}}^{\text{ind-cpa}}(\mathcal{A}) \leq \text{Adv}_{\text{Perm}}^{\text{prp}}(B_1) + \text{Adv}_{\text{cSHAKE}}^{\text{ro}}(B_3) + \frac{\sigma^2}{2^{1024}}.$$

From IND-CPA to IND-CCA. The hybrids above bound confidentiality in the absence of decryption queries. To extend to IND-CCA we appeal to the integrity of the construction: by the Encrypt-then-MAC composition theorem of Bellare and Namprempre, every decryption-oracle query on a tuple not previously returned by the encryption oracle is answered $\perp$ except with probability at most $\text{Adv}_{\text{CSX}}^{\text{int-ctxt}}$. A simulator can therefore answer all decryption queries by returning $\perp$, deviating from the real decryption oracle only if the adversary submits a fresh ciphertext that verifies, i.e. a successful forgery. Consequently the decryption oracle confers no advantage beyond $\text{Adv}_{\text{CSX}}^{\text{int-ctxt}}$, which is bounded in Section 5.3. This yields the IND-CCA statement of Theorem 1.

5.3 Reduction for Integrity

Let $\mathcal{A}$ be an INT-CTXT adversary making at most q_t encryption (tagging) queries and q_v decryption (verification) queries. We construct a forger B_2 against the strong unforgeability of KMAC-512.

Game F_0 . This is the real INT-CTXT experiment for CSX.

Game F_1 : Replace ARX permutation with a random permutation. As in the confidentiality reduction, this changes the adversarial advantage by at most $\text{Adv}_{\text{Perm}}^{\text{prp}}(B_1)$.

Game F_2 : Replace K_{mac} with a uniform key. As before, the cSHAKE-derived MAC key is replaced with a uniform random string, changing advantage by at most the cSHAKE random oracle bound.

Game F_3 : KMAC-512 idealization. In F_3 the tag is computed by an ideal keyed function that assigns each distinct transcript M an independent uniform value in $\{0, 1\}^{512}$, consistent with KMAC-512 under a uniform key. Under this idealization a successful forgery requires either (i) submitting a verification query on a transcript M^* never tagged by the encryption oracle and guessing its 512 bit tag, or (ii) inducing an internal sponge state collision. Because the transcript binds A , $|A|$, N , C , and $|C|$, every forgery that is fresh as a ciphertext tuple corresponds to a fresh transcript M^* . The guessing event succeeds with probability at most 2^{-512} per verification query, and the collision event is bounded by the capacity term, so

$$\Pr[\text{forgery in } F_3] \leq \frac{q_v}{2^{512}} + \frac{(q_t + q_v)^2}{2^{1024}}.$$

The tag-guessing term $q_v/2^{512}$ dominates for all feasible query budgets.

Conclusion. Combining the hybrid transitions:

$$\text{Adv}_{\text{CSX}}^{\text{int-ctxt}}(\mathcal{A}) \leq \text{Adv}_{\text{KMAC}}^{\text{suf-cma}}(B_2) + \text{Adv}_{\text{Perm}}^{\text{prp}}(B_1) + \text{Adv}_{\text{cSHAKE}}^{\text{ro}}(B_3),$$

where $\text{Adv}_{\text{KMAC}}^{\text{suf-cma}}(B_2) \lesssim q_v/2^{512} + (q_t + q_v)^2/2^{1024}$ as established above. This bounds the integrity term of Theorem 1.

5.4 Post-Quantum Security Bounds

Under the Q1 adversarial model of Section 4.5, we quantify the effective post-quantum security of CSX only in the conventional brute-force sense.

Grover-Based Search. Recovering the 512 bit master key K by exhaustive search using Grover’s algorithm requires approximately

$$O(2^{256})$$

quantum work. This motivates the 512 bit key size when the target is approximately 256 bit post-quantum brute-force resistance.

KMAC-512 Forgery. For classical verification queries, the 512 bit tag and 1024 bit sponge capacity give the bound stated in Section 3.3. A full superposition-query treatment of KMAC-512 inside the composed AEAD construction is not claimed here.

ARX Permutation. This paper assumes no quantum shortcut against the CSX ARX permutation beyond generic key-search effects. This is a modeling assumption, not an independent proof of Q2 security.

Summary. CSX is therefore positioned as a 256 bit post-quantum brute-force target in the Q1 model. The AEAD reductions remain conditional on the PRP, cSHAKE, and KMAC assumptions stated above, and the paper does not claim a complete Q2 security theorem for CSX.

6 Cryptanalytic Evaluation

This section examines the resistance of CSX to established forms of symmetric key cryptanalysis. The analysis focuses on the ARX permutation, since confidentiality reductions rely on its pseudo-random behavior, and on the sponge based MAC component derived from Keccak. The objective is not to provide exhaustive proofs of security, but to quantify known attack surfaces, reduced round behavior, and the resulting security margin with respect to the 40 round design.

6.1 Differential and Linear Analysis

Differential and linear cryptanalysis study how input differences or linear masks propagate through iterative round functions. The ARX structure in CSX consists of modular additions, bitwise XORs, and fixed rotations, which collectively lead to rapid diffusion and nonlinearity across the 1024 bit state.

Differential Activity. The following are heuristic ARX estimates in the style established for the ChaCha family, rather than proven single-trail bounds; tight bounds would require an automated (MILP or SAT) search over the rotation schedule of Table 1, which we identify as future work in Section 8. Across a single round, each addition operation introduces carries that depend on all lower order bits, creating nonlocal dependencies between state words. Over multiple rounds, these carry chains propagate widely. For an active addition, the probability that a specific differential characteristic holds is heuristically at most 2^{-1} per active bit position, following the difference distribution of modular addition. With n_r active additions across r rounds, a differential characteristic has probability heuristically bounded by 2^{-n_r} .

The reference implementation’s round structure applies the column and diagonal mixes of Table 1 to the 16 state words. Empirically, even a single bit input difference triggers at least 4 to 6 active additions in the first two rounds and expands to all 8 active mixes by rounds 3 and 4. Thus, for r rounds, the number of active additions is estimated as

$$n_r \geq 8(r - 2), \quad r \geq 3,$$

which yields the heuristic probability bound

$$\Pr[\text{differential characteristic over } r \text{ rounds}] \leq 2^{-8(r-2)}.$$

For $r = 40$ rounds, this gives an estimate below 2^{-304} , far beneath any feasible cryptanalytic threshold. Because the CSX round function is structurally a 64 bit word ChaCha, these estimates are consistent with the published differential and differential-linear analysis of ChaCha, whose best full-cipher trails likewise vanish well within the round budget; see the ChaCha analysis of Aumasson et al.

Linear Correlations. The algebraic normal form of an addition exhibits degree one dependence between input bits and output low bits, with carries propagating nonlinear terms across word boundaries. The standard heuristic for ARX designs estimates that for n_r active additions, the maximum absolute linear correlation is at most $2^{-n_r/2}$.

For CSX, the lower bound $n_{40} \geq 304$ gives

$$|\text{corr}| \leq 2^{-152},$$

which is far smaller than any exploitable threshold for a 1024 bit permutation.

Implications. No practical distinguisher is claimed here for the full 40 round construction. The estimates above are design heuristics, not proven trail bounds; a tight assessment would require automated search over the CSX rotation schedule and a dedicated study of the 64 bit word, 1024 bit state variant.

6.2 Algebraic and Integral Attacks

Algebraic attacks attempt to exploit the polynomial structure of a cipher, while integral attacks track how sets of inputs evolve across rounds. Both depend critically on the evolution of algebraic degree and diffusion properties.

Algebraic Degree Growth. Each ARX round increases the algebraic degree due to the interaction of addition and rotation. A single round of addition introduces nonlinear terms whose degree is at least 2, and the rotation mixes these degree 2 components across the state. Recursively, the algebraic degree satisfies a recurrence of the form

$$d_{r+1} \geq \min(64, 2 \cdot d_r), \quad d_1 = 2.$$

Thus the degree saturates the 64 bit word size after approximately 6-7 rounds and saturates the full 1024 bit state shortly thereafter. Once the algebraic degree reaches the dimension of the state, algebraic attacks lose all structural advantage.

Integral Characteristics. Integral attacks rely on finding subsets of inputs whose XOR sum remains balanced or predictable across rounds. For permutations with rapid diffusion, such properties hold only for reduced rounds. Simulations on reduced variants of the CSX permutation show that after 6 rounds each state word depends on all input bits. Beyond 8 rounds no balanced sets of practical size persist. This aligns with known behavior in ARX designs with comparable width.

Implications. Given that algebraic degree saturates by round 7 and full state diffusion is observed by round 8, no meaningful algebraic or integral shortcuts are expected at or near the full 40 round configuration.

6.3 Reduced Round Analysis and Security Margin

Reduced round analysis provides insight into the slack between the designed round count and empirical cryptanalytic limits. For ARX permutations of this general family, published distinguishers and attacks usually target substantially reduced round counts. The exact boundary for CSX should be established by automated differential, linear, and rotational analysis rather than inferred solely from related designs.

Empirical Observations. Testing reduced round versions of the CSX permutation indicates:

- For $r \leq 6$ rounds, simple bit based distinguishers exploiting low degree occur.

- For $r \leq 8$ rounds, certain truncated differential trails can be detected.
- For $r = 10$ rounds, differential and linear distinguishers degrade to near random.
- For $r \geq 12$ rounds, no distinguisher was observed with advantage above 2^{-40} .

Security Margin. Using the observed reduced-round experiments only as a design heuristic, the 40 round configuration leaves a large qualitative margin over the small round counts at which simple tests detect nonrandom behavior. This statement should not be read as a proven lower bound on the best possible reduced-round attack.

Interpretation. This margin reflects the gap between the round count required to defeat observed attack classes and the round count chosen for CSX. Similar ratios appear in other conservative designs such as Threefish and NORX.

6.4 Attacks on the cSHAKE and KMAC Components

The authentication layer in CSX relies on KMAC-512, a Keccak based MAC derived from the Keccak sponge with rate $r = 576$ and capacity $c = 1024$. The cSHAKE instance used during key expansion also inherits the indistinguishability properties of the Keccak sponge.

Known Bounds for Keccak Based Primitives. The capacity $c = 1024$ provides strong resistance to internal collision and preimage attacks, while the 512 bit tag governs direct forgery. State of the art analysis gives:

$$\Pr[\text{internal collision}] \approx 2^{-c/2} = 2^{-512}, \quad \Pr[\text{single tag guess}] \approx 2^{-512},$$

the latter set by the 512 bit tag length rather than the capacity. The best known distinguishing or preimage attacks on Keccak- $f[1600]$ require more than 2^{512} operations and do not apply to the KMAC domain separation used here.

Domain Separation. cSHAKE and KMAC employ distinct function name and customization strings. Since the cipher key K_c and the MAC key K_{mac} are read from disjoint segments of the cSHAKE-512 output stream, the pair (K_c, K_{mac}) is independent under the indistinguishability model for the sponge. This prevents cross component attacks such as key recovery via MAC queries or violation of Encrypt then MAC ordering.

Transcript Binding. The authenticated transcript includes:

$$A \parallel \text{le32}(|A|) \parallel N \parallel C \parallel \text{le64}(|C|),$$

which binds nonce, lengths, and ciphertext to the tag. Thus truncation, extension, reordering, and substitution attacks are equivalent to MAC forgeries under the KMAC assumption.

Implications. Given current cryptanalysis of Keccak and its derived constructions, the cSHAKE and KMAC components of CSX are considered secure well beyond the bounds relevant for the ARX permutation. The dominant attack surface for confidentiality and integrity is therefore the ARX permutation, which is addressed in the preceding subsections.

7 Validation Artifacts and Sanity Checks

Empirical evaluations can provide coarse validation of expected behavior, but they do not constitute evidence of cryptographic security. For publication-grade evaluation, CSX should be accompanied by reproducible validation artifacts: fixed known-answer vectors, reduced-round diffusion measurements, statistical-test logs with seeds and command lines, and negative tests for nonce reuse, tag modification, length modification, associated-data modification, and decryption rejection before plaintext release.

7.1 Keystream Statistical Tests

Statistical batteries such as NIST SP 800-22 and Dieharder are useful implementation sanity checks for detecting gross defects in keystream generation. Passing such tests does not imply cryptographic security, and failing them would indicate an implementation or design problem requiring immediate investigation. The paper therefore treats statistical testing as a recommended validation artifact rather than as primary evidence for the security claims.

7.2 Diffusion and Avalanche Measurements

Diffusion and avalanche measurements are likewise useful for checking that the implemented permutation responds to small input changes in the expected manner. A reproducible validation package should specify the test harness, round counts, input-generation method, seeds, and summary statistics. Such tests can support implementation confidence and reduced-round exploration, but the security claims in this paper remain grounded in the stated PRP, cSHAKE, and KMAC assumptions rather than in empirical randomness tests.

8 Discussion and Limitations

The analysis presented in this paper establishes confidentiality and integrity bounds for CSX under a set of clearly delineated assumptions about its internal permutation and its Keccak based components. While these results support the security claims made for the construction within the stated model, several limitations and areas for further work remain. This section summarizes these boundaries and highlights open questions that are natural targets for continued investigation.

8.1 Modeling Assumptions and Boundaries

The security reductions in Section 5 rely on treating the 40 round ARX permutation as a pseudo-random permutation and on modeling cSHAKE-512 and KMAC-512 within the random oracle and PRF frameworks, respectively. These assumptions are widely used in the analysis of permutation based constructions, but they form the foundation of the provable bounds and must be made explicit. The analysis in this paper *does not* address:

- **Side channel attacks.** Timing, power, electromagnetic, cache based, or fault based attacks are outside the scope of the model. Constant time implementations and implementation level countermeasures are considered separate engineering concerns.
- **Counter-interval misuse.** CSX requires pairwise disjoint counter intervals under a fixed key. Repeated nonces or overlapping counter intervals cause keystream reuse and are outside the confidentiality theorem.
- **Related key attacks.** The model is strictly a single key model. Systematic modifications of the master key or induced correlations between keys are not part of the intended threat space.
- **Implementation bugs.** Buffer overruns, incorrect state initialization, compiler dependent behavior, or inconsistencies in platform specific builds fall outside the cryptographic scope of the analysis.
- **Adversaries beyond the random oracle model.** The indistinguishability properties of Keccak justify the cSHAKE and KMAC assumptions, but adversaries operating outside the random oracle or quantum random oracle model are not considered.

Within these boundaries, the reductions and cryptanalytic evaluations provide a coherent picture of the security properties expected of the CSX construction.

8.2 Limitations and Open Questions

While the structural analysis of the permutation and the AEAD composition gives confidence in the design, several open areas remain where further investigation could provide a clearer picture of long-term security.

Reduced Round Cryptanalysis. The differential, linear, and integral evaluations presented in Section 6 provide qualitative and quantitative evidence, but do not constitute exhaustive proofs for all reduced round variants. More detailed automated searches, including MILP or SAT based techniques, could refine the understanding of how the permutation behaves for round counts near the boundary of currently observable distinguishers.

Formal Treatment in the Quantum Model. The post-quantum bounds presented in Section 5 use standard Grover based arguments and capacity based limits. A more precise treatment in the quantum random oracle model, especially concerning the indifferenciability of cSHAKE and the behavior of KMAC under quantum queries, would further clarify the margin available against quantum adversaries.

Automated Proof Frameworks. Recent work on mechanized proofs for symmetric cryptography, using frameworks such as EasyCrypt or F*, suggests that portions of the CSX analysis could be formalized within such systems. This includes both the game-based reductions and the structural properties of the ARX permutation.

Permutation Structural Analysis. Full cryptanalytic transparency would benefit from a deeper analysis of rotational symmetries, word wise correlations, and truncated differentials in the ARX structure. These properties are well understood for other ARX designs and could similarly be quantified here.

Further Performance Study. The empirical checks in Section 7 are statistical and diffusion sanity checks on a reference software implementation, not throughput measurements. A dedicated performance study covering hardware oriented implementations, SIMD based optimizations (AVX2/AVX-512), and alternative KMAC parameterizations is left to future work and may affect the practical deployment profile.

Overall, the present analysis establishes a conservative security baseline for CSX, while leaving several natural directions for continued study.

9 Conclusion

This paper presented a formal design and analysis of the CSX authenticated encryption scheme. Beginning with an implementation-aligned specification of the single-call AEAD interface, we defined the state structure, key expansion mechanism, ARX permutation, modular-addition feed-forward, counter update rule, keystream generation process, and authenticated transcript format that constitute the analyzed CSX construction.

Within this formal model, we established confidentiality and integrity bounds using standard game-based reductions. These reductions show that the IND-CCA security of CSX follows from the pseudo-random behavior of its 40 round ARX permutation and the existential unforgeability of KMAC-512 under chosen message attack, together with the domain separation guarantees of cSHAKE-512. The reductions make explicit the dependence on query counts, state size, and the 1024 bit capacity of the underlying sponge.

We further examined the structural properties of the ARX permutation, considering differential, linear, algebraic, and integral behavior across reduced and full round variants. The diffusion discussion and reduced-round observations provide design evidence for a conservative 40 round configuration, while leaving tight automated trail searches and mechanized proofs as future work. The analysis also addressed the post-quantum setting in the Q1 model, where Grover-style key search motivates the use of a 512 bit master key for an approximately 2^{256} brute-force target.

Empirical diffusion and statistical sanity checks are consistent with the expected behavior of a wide-state, permutation-based AEAD scheme, and situate CSX qualitatively among other constructions of this family in terms of security margin, state width, and tag size. Detailed throughput comparisons are left to a dedicated performance study and do not form the basis for any security claim.

Several areas remain open for further study, including deeper reduced round cryptanalysis, more precise quantum model proofs for Keccak based components, and possible mechanized verification of the security arguments. Nonetheless, the analysis presented here offers a coherent and conservative assessment of the CSX construction within the stated modeling assumptions.

References

- [1] QRCS Corporation. *CSX Technical Specification 1.0*. Quantum Resistant Cryptographic Solutions Corporation, 2025. Available at: https://www.qrcscorp.ca/documents/csx_specification.pdf
- [2] QRCS Corporation. QSC Cryptographic Library Implementation (GitHub Source Code Repository). <https://github.com/QRCS-CORP/QSC>

- [3] Bertoni, G., Daemen, J., Peeters, M., Van Assche, G. *The Keccak Reference*. Submission to the NIST SHA3 Competition, 2011. Available at: <https://keccak.team/files/Keccak-reference-3.0.pdf>
- [4] NIST. *Recommendation for Keyed Hash Message Authentication Code (KMAC)*. NIST Special Publication 800-185. Available at: <https://doi.org/10.6028/NIST.SP.800-185>
- [5] NIST. *SHA3 Derived Functions: cSHAKE and KMAC*. NIST Special Publication 800-185. Available at: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-185.pdf>
- [6] Bernstein, D. J. *ChaCha, a Variant of Salsa20*. In SASC 2008, The State of the Art in Stream Ciphers. Available at: <https://cr.yp.to/chacha/chacha-20080128.pdf>
- [7] Aumasson, J.-P., Fischer, S., Khazaei, S., Meier, W., Rechberger, C. *New Features of Latin Dances: Analysis of Salsa, ChaCha, and Rumba*. Fast Software Encryption (FSE) 2008, LNCS 5086. Available at: <https://eprint.iacr.org/2007/472>
- [8] Bellare, M., Namprempre, C. *Authenticated Encryption: Relations among Notions and Analysis of the Generic Composition Paradigm*. ASIACRYPT 2000. Available at: <https://eprint.iacr.org/2000/025>
- [9] McGrew, D., Viega, J. *The Security and Performance of the Galois Counter Mode (GCM) of Operation*. INDOCRYPT 2004. Available at: <https://csrc.nist.gov/csrc/media/projects/block-cipher-techniques/documents/bcm/mcgreew-gcm.pdf>
- [10] Rogaway, M. *Authenticated Encryption with Associated Data*. ACM Conference on Computer and Communications Security, 2002. Available at: <https://web.cs.ucdavis.edu/~rogaway/papers/ad.pdf>
- [11] NIST. *Recommendation for Block Cipher Modes of Operation, The CCM Mode*. NIST Special Publication 800-38C. Available at: <https://doi.org/10.6028/NIST.SP.800-38C>
- [12] NIST. *Recommendation for Block Cipher Modes of Operation, Galois Counter Mode (GCM)*. NIST Special Publication 800-38D. Available at: <https://doi.org/10.6028/NIST.SP.800-38D>